University of Engineering & Technology Peshawar

Computer Programming (C++) Lab

Lab 11

STUDENT NAME	MUHAMMAD SAHAL AYUB
REGISTRATION NO:	4042F24
COUSRE:	EE-170LCOMPUTER PROGRAMMING LAB
DATE:	14/05/26
Lab Title:	STRING IN C++

SUBMITTED TO ENGR MUHAMMAD RIZWAN

1 Objective:

The objective of this lab session is to introduce and apply the following concepts in C++:

- C-String (C-style strings)
- Declaration and Initialization of C-Strings
- String Objects using the Standard C++ Library

2 Introduction to C++ Strings:

A string is a collection of characters. There are two types of strings commonly used in C++:

- C-strings (C-style Strings)
- Strings that are objects of the string class (Standard C++ Library)

2.1 C-Strings:

In C programming, the collection of characters is stored in the form of arrays. This is also supported in C++, hence the name C-strings. C-strings are arrays of type char terminated with a null character (\0). The ASCII value of the null character is 0.

2.1.1 Declaration and Initialization

The following examples demonstrate the various ways to declare and initialize a C-string:

```
char myString[] = "Hello";              // Automatically adds \0
char myString[6] = "Hello";             // Explicit size
char myString[] = {'H','e','l','l','o','\0'};  // Character array
char myString[100] = "Hello";           // Pre-allocating extra space
```

Note: The C++ compiler automatically places \0 at the end of the string constant. Unlike arrays, it is not necessary to use all the allocated space.

2.2 Reading Strings:

Example 1: Basic string input with cin:

```
#include <iostream>  // for input/output operations
using namespace std;
int main() {
    // Declare a character array to store the string
    char str[100];
    // Prompt user to enter a string
    cout << "Enter a string: ";
    // Read string using extraction operator >>
    cin >> str;
    // Display the entered string
    cout << "You entered: " << str << endl;
    // Prompt for another string
    cout << "Enter another string: ";
    // Read another string
    cin >> str;
    // Display the second string
    cout << "You entered: " << str << endl;
    return 0;
```

```
}
```

Output

```
Enter a string: Department
You entered: Department
Enter another string: Electrical Engineering
You entered: Electrical
```

Notice: The extraction operator >> with cin considers a space " " as a terminating character. Hence, only "Electrical" is displayed instead of "Electrical Engineering".

Example 2: Reading a full line with cin.get():

```
#include <iostream>  // for input/output operations
using namespace std;
int main() {
    // Declare a character array to store the string
    char str[100];
    // Prompt user to enter a string
    cout << "Enter a string: ";
    // Read up to 100 characters including spaces
    cin.get(str, 100);
    // Display the entered string
    cout << "You entered: " << str << endl;
    return 0;
}
```

Output

```
Enter a string: Electrical Engineering
You entered: Electrical Engineering
```

The cin.get() function takes two arguments: the name of the string (address of first element) and the maximum size of the array. This allows reading strings containing blank spaces.

2.3 String Objects:

In C++, you can also create a string object for holding strings. Unlike char arrays, string objects have no fixed length and can be extended as required.

```
#include <iostream>  // for input/output operations
#include <string>    // for string class
using namespace std;
int main() {
    // Declare a string object
    string str;
    // Prompt user to enter a string
    cout << "Enter a string: ";
    // Read a full line including spaces using getline()
    getline(cin, str);
    // Display the entered string
    cout << "You entered: " << str << endl;
    return 0;
}
```

Output

```
Enter a string: University of Engineering and Technology
You entered: University of Engineering and Technology
```

The getline() function takes the input stream (cin) as the first parameter and the string variable as the second, allowing full-line input including spaces.

3 String Functions:

C++ supports a wide range of functions that manipulate null-terminated strings. The table below summarizes the most commonly used ones:

Function	Example	Description
strcpy()	`strcpy(s1, s2);`	Copies string s2 into string s1.
strcat()	`strcat(s1, s2);`	Concatenates string s2 onto the end of string s1.
strlen()	`strlen(s);`	Returns the length of string s.
strcmp()	`strcmp(s1, s2);`	Returns 0 if s1==s2; <0 if s1<s2; >0 if s1>s2.
strrev()	`strrev(s);`	Reverses the given string.
strlwr()	`strlwr(s);`	Converts string to lowercase.
strupr()	`strupr(s);`	Converts string to uppercase.

4 Lab Tasks — Lab 11:

Task 1: String Length Calculation:

Write a C++ program that declares and initializes a string variable called message with a phrase of your choice and uses a built-in function to calculate and display the length of the string.

```
#include <iostream>  // for input/output operations
#include <cstring>   // for string functions like strlen()
using namespace std;
int main() {
    // Declare and initialize a string variable
    char message[] = "UET Peshawar - Engineering Excellence";
    // Calculate the length of the string using strlen()
    int len = strlen(message);
    // Display the message
    cout << "Message: " << message << endl;
    // Display the length of the string
    cout << "Length of the string: " << len << endl;
    return 0;
}
```

Output

```
Message: UET Peshawar - Engineering Excellence
```

```
Length of the string: 37
```

Task 2: String Concatenation:

Declare two string variables called firstName and lastName, initialize them with your first and last name, concatenate them to form a full name, and display it.

```
#include <iostream>  // for input/output operations
#include <cstring>   // for string functions like strcpy() and strcat()
using namespace std;
int main() {
    // Declare and initialize first name string
    char firstName[50] = "Ali";
    // Declare and initialize last name string
    char lastName[50] = "Khan";
    // Declare full name string to store concatenated result
    char fullName[100];
    // Copy first name to full name
    strcpy(fullName, firstName);
    // Concatenate a space to full name
    strcat(fullName, " ");
    // Concatenate last name to full name
    strcat(fullName, lastName);
    // Display first name
    cout << "First Name: " << firstName << endl;
    // Display last name
    cout << "Last Name: " << lastName << endl;
    // Display the full concatenated name
    cout << "Full Name: " << fullName << endl;
    return 0;
}
```

Output

```
First Name: Ali
Last Name: Khan
Full Name: Ali Khan
```